



Meta-Sezgisel Algoritmalar

(Genetik Algoritma)

OPTİMİZASYON

11.Hafta

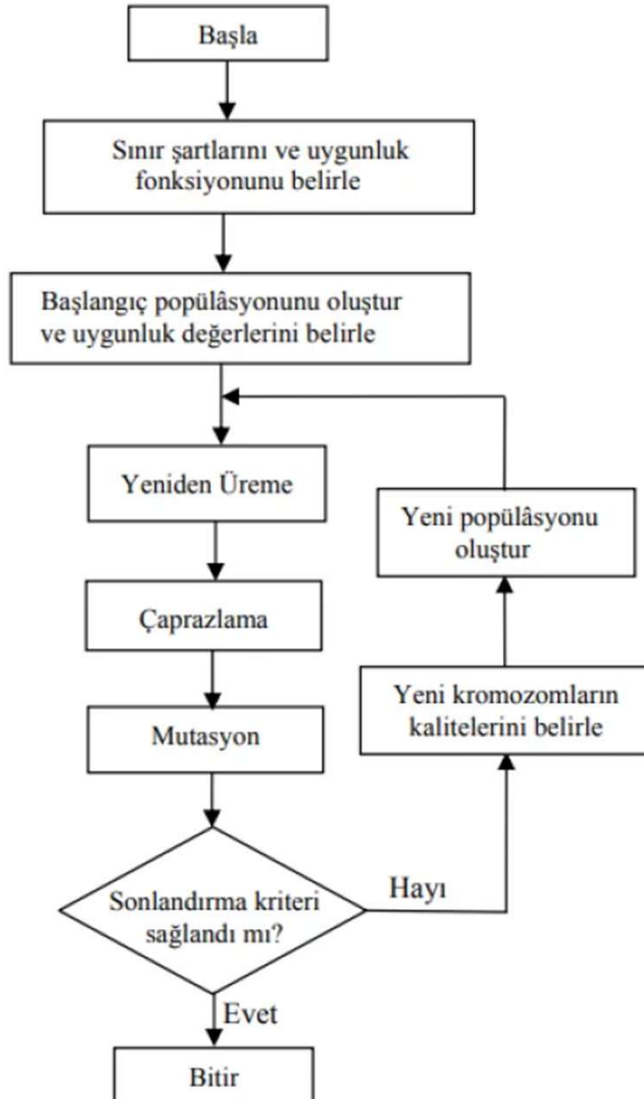
Dr. Furkan YENER

Genetik Algoritma Temel Kavramlar

Genetik Algoritma (Genetic Algorithm - GA), Charles Darwin'in **doğal seçim** ve **genetik miras** teorilerinden ilham alan, evrimsel hesaplama (evolutionary computation) çatısı altında yer alan bir **meta-sezgisel optimizasyon ve arama algoritmasıdır**. Doğada canlı popülasyonlarının nesiller boyunca çevreye adapte olarak iyileşmesi prensibini taklit eder. GA, özellikle türev alınamayan, çok modlu, gürültülü veya kesikli çözüm uzayına sahip karmaşık problemlerde global optimumu bulmak için kullanılır.

Bileşen	Biyolojik Karşılığı	Algoritmik Rolü
Kromozom (Birey)	DNA dizisi	Bir çözüm adayının kodlanmış temsilidir (genellikle bit dizisi, dizi, ağaç veya gerçek sayı vektörü).
Popülasyon	Canlılar topluluğu	Çözüm adaylarının oluşturduğu küme.
Uygunluk Fonksiyonu (Fitness)	Doğada hayatta kalma/üreme başarısı	Her çözümün kalitesini nicel olarak ölçen matematiksel fonksiyon. Maximization veya minimization amaçlıdır.
Seçim (Selection)	Doğal seçim	Daha yüksek fitness değerine sahip bireylerin üreme havuzuna alınma olasılığının artırılması (Rulet, Turnuva, Rank vb.).
Çaprazlama (Crossover)	Genetik rekombinasyon	İki ebeveynin genetik materyalinin birleştirilerek yeni yavrular oluşturulması (Tek noktalı, çok noktalı, uniform, aritmetik vb.).
Mutasyon (Mutation)	DNA replikasyon hataları	Çözüm uzayında çeşitliliği korumak ve yerel optimumlardan kaçmak için genlerde rastgele küçük değişiklikler yapma.
Elitizm	-	En iyi birey(ler)in değiştirilmeden bir sonraki nesile aktarılması, çözüm kalitesinin düşmesini engeller.

Genetik Algoritma Akış Şeması



Algorithm 1 The pseudo-code of genetic algorithm (GA).

```
1: begin GA
2: create initial population
3: while iteration_count <  $K$  ( $K$  = max. number of iteration) do
4:   selection and elitism
5:   crossover
6:   mutation
7:   increment iteration_count
8: end while
9: output best chromosome
10: end GA
```

Genetik Algoritma Örnek Problem

Senaryo:

Bir bilişim sisteminin işlemci yük dengeleme parametresi (x) optimize edilmek istenmektedir. Sistemin performans metrikleri, x tamsayı değeri için $f(x) = -x^2 + 10x$ fonksiyonu ile modellenmiştir. Amaç, bu fonksiyonun maksimum değerini veren x parametresini Genetik Algoritma kullanarak bulmaktır.

Problem Tanımı:

- **Arama Uzayı:** $x \in [0, 15]$
- **Kodlama:** 4-bit ikili (binary) gösterim (0000 $\rightarrow$ 0, 1111 $\rightarrow$ 15)
- **Uygunluk Fonksiyonu (Fitness):** $F(x) = -x^2 + 10x$
- **Popülasyon Boyutu:** 4 birey
- **Başlangıç Popülasyonu:**

B1 = 0110 | B2 = 1010 | B3 = 0011 | B4 = 1100



Genetik Algoritma Örnek Problem

Birey	İkili	x	$F(x) = -x^2 + 10x$	Fitness
B1	0110	6	$-36 + 60 = 24$	24
B2	1010	10	$-100 + 100 = 0$	0
B3	0011	3	$-9 + 30 = 21$	21
B4	1100	12	$-144 + 120 = -24$	0
Toplam				**45**

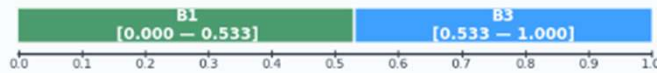
Genetik Algoritma Örnek Problem

2. Rulet Çarkı Seçimi:

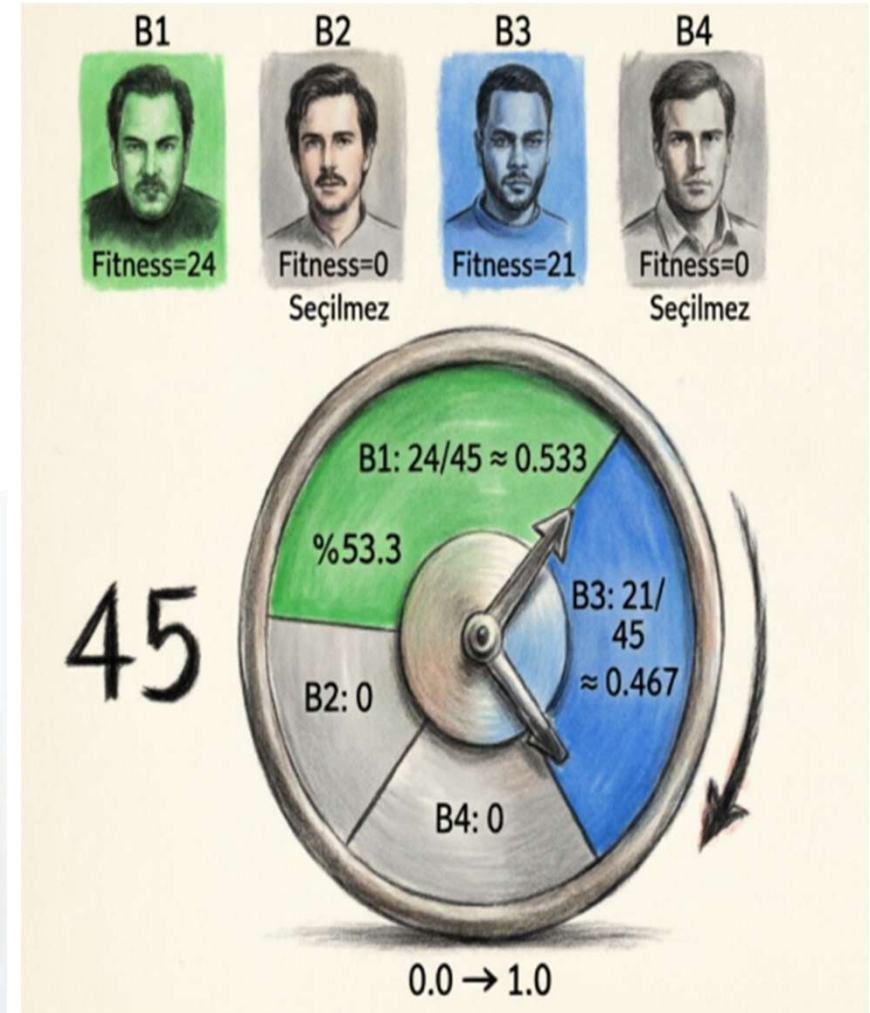
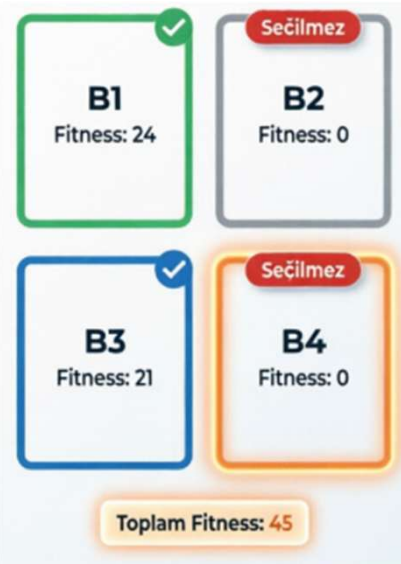
- Olasılıklar: $P(B1)=24/45 \approx 0.533$, $P(B2)=0$, $P(B3)=21/45 \approx 0.467$, $P(B4)=0$
- Birikimli Olasılık: $C1=0.533$, $C2=0.533$, $C3=1.000$, $C4=1.000$
- $r_1=0.32 \rightarrow \leq 0.533 \rightarrow B1$
- $r_2=0.67 \rightarrow > 0.533 \ \& \ \leq 1.0 \rightarrow B3$
- $r_3=0.89 \rightarrow B3$
- $r_4=0.15 \rightarrow B1$
- Seçilen Ebeveynler: (B1, B3) ve (B3, B1)

3. RASTGELE SAYI İLE SEÇİM

Birikimli Olasılık Aralıkları:



- 1 $r_1 = 0.32 \rightarrow 0.32 \in [0.000, 0.533] \rightarrow B1$ seçildi
- 2 $r_2 = 0.67 \rightarrow 0.67 \in [0.533, 1.000] \rightarrow B3$ seçildi
- 3 $r_3 = 0.89 \rightarrow 0.89 \in [0.533, 1.000] \rightarrow B3$ seçildi
- 4 $r_4 = 0.15 \rightarrow 0.15 \in [0.000, 0.533] \rightarrow B1$ seçildi

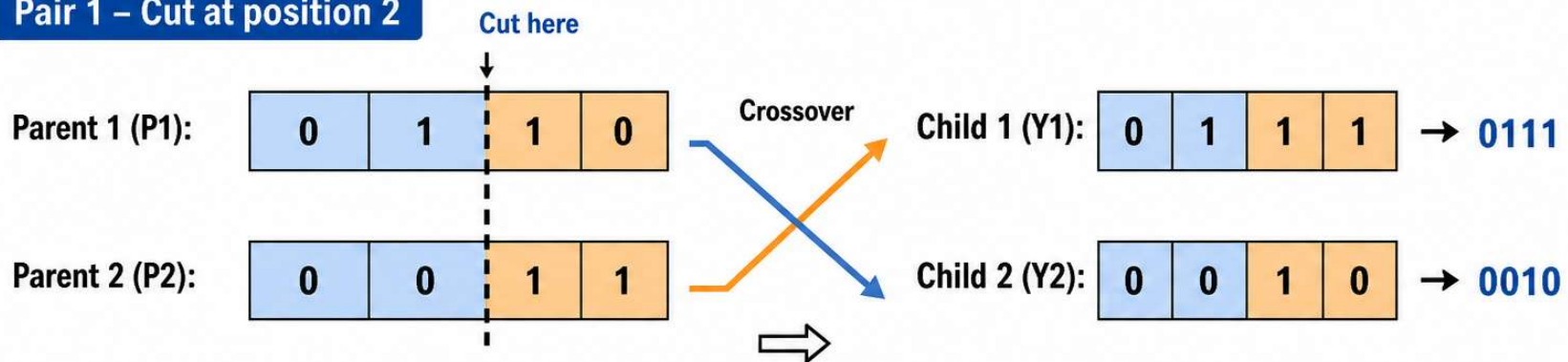


Genetik Algoritma Örnek Problem

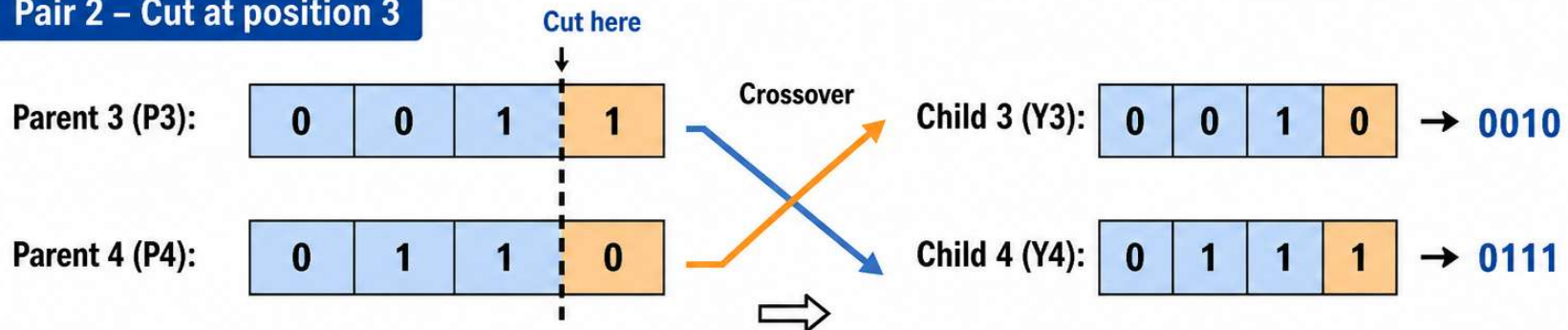
3. Çaprazlama (Tek Nokta):

- Çift 1 (Kesim @2): $01|10 \otimes 00|11 \rightarrow Y1=0111, Y2=0010$
- Çift 2 (Kesim @3): $001|1 \otimes 011|0 \rightarrow Y3=0010, Y4=0111$

Pair 1 – Cut at position 2



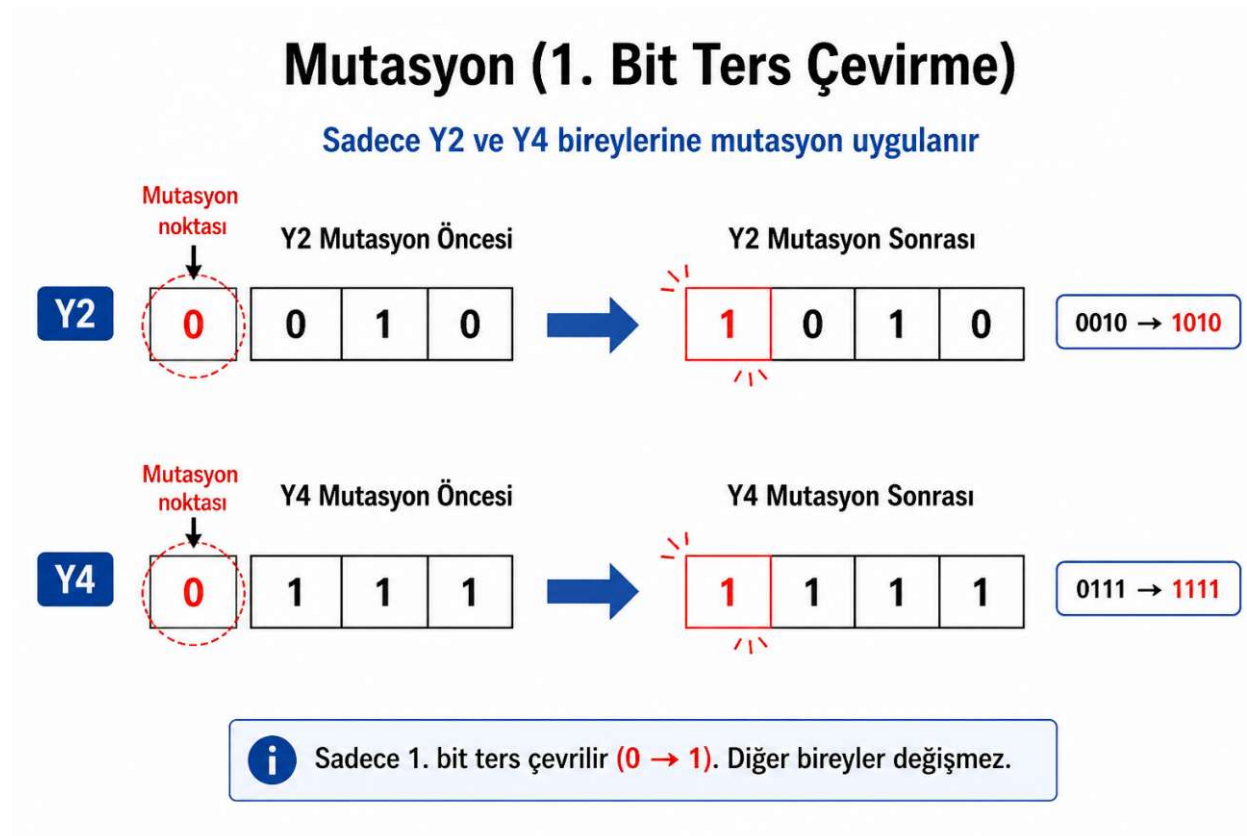
Pair 2 – Cut at position 3



Genetik Algoritma Örnek Problem

4. Mutasyon (1. bit ters çevirme Y2 ve Y4'te):

- Y2 = 0010 → 1. bit 0 → 1 → 1010
- Y4 = 0111 → 1. bit 0 → 1 → 1111
- Diğer bireyler değişmez.



Genetik Algoritma Örnek Problem

Birey	İkili	x	F(x)
N1	0111	7	21
N2	1010	10	0
N3	0010	2	16
N4	1111	15	0

Gözlem: Ortalama uygunluk başlangıçta $45/4 = 11.25$, 1. iterasyon sonunda $37/4 = 9.25$ 'e düşmüştür. Bu durum, GA'nın tek iterasyonda monoton artış garantisi vermediğini, yerel optimumlardan kaçınma yeteneğini ve **elitizm** mekanizmasının neden gerekli olduğunu tartışmak için mükemmel bir pedagojik fırsattır.

Elitizm, genetik algorithmada bir nesilden sonraki nesile, **en iyi fitness değerine sahip birey(ler)**in hiçbir değişikliğe uğramadan (çaprazlama veya mutasyon olmadan) doğrudan aktarılması stratejisidir.

Genetik Algoritma Elitizim

Genetik algorithmda seçim, çaprazlama ve mutasyon operatörleri **stokastik** (rastgelelik içeren) süreçlerdir. Bu durum şu riskleri doğurur:

Risk

Açıklama

✗ En iyi çözümün kaybolması

Yüksek fitness'lı bir birey, seçimde şans eseri seçilmeyebilir veya çaprazlama/mutasyonla bozulabilir.

✗ Fitness değerinde gerileme

Yeni nesil, önceki nesilden daha kötü çözümler içerebilir.

✗ Yakınsama yavaşlaması

Algoritma, zaten keşfedilmiş iyi bölgeleri tekrar tekrar aramak zorunda kalabilir.

Genetik Algoritma Elitizim

1. Mevcut popülasyonun fitness değerlerini hesapla.
2. En iyi 'k' bireyi belirle (elit sayısı, genellikle 1-5 arası).
3. Bu elit bireyleri kopyala ve "elit havuzu"na al.
4. Geri kalan (N-k) bireyi normal GA operatörleriyle üret:
 - Seçim → Çaprazlama → Mutasyon
5. Yeni nesil = [Elit havuzu] + [Yeni üretilen bireyler]
6. Bir sonraki nesile geç.

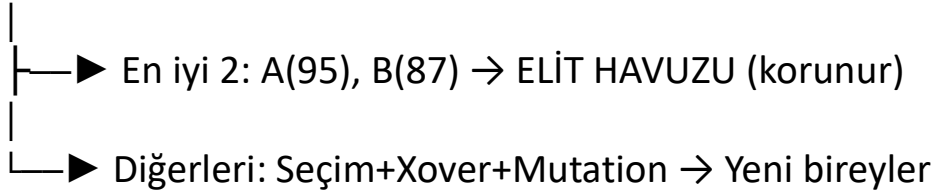
Spor Takımı Analojisi:

Bir futbol takımı düşünün. Sezon boyunca en başarılı 2-3 oyuncu (elitler), yeni sezonda **kesinlikle kadroda tutulur**. Diğer oyuncular için ise denemeler yapılır, yeni transferler alınır. Bu sayede:

- Takımın çekirdek kalitesi korunur
- Yeni yetenekler keşfedilir
- Ancak çok fazla "garanti oyuncu" tutarsanız, takım yenilenemez ve gerileyebilir

Genetik Algoritma Elitizim Çeşitleri

Nesil t: [A(95), B(87), C(82), D(76), ...] $\leftarrow$ Fitness değerleri



Nesil t+1: [A(95), B(87), E(91), F(84), G(79), ...] $\leftarrow$ Elitler + Yeniler

Tür	Açıklama	Kullanım Senaryosu
Saf Elitizm	Sadece en iyi birey(ler) korunur, diğerleri tamamen yenilenir.	Basit problemler, hızlı prototipleme.
Kademeli Elitizm	Elit sayısı nesil ilerledikçe artırılır/azaltılır.	Dinamik veya çok modlu problemler.
Adaptif Elitizm	Elit sayısı, popülasyon çeşitliliğine göre otomatik ayarlanır.	Karmaşık, değişken fitness landscape'leri.
Çok Amaçlı Elitizm	Pareto-optimal çözümler elit havuzunda tutulur (NSGA-II vb.).	Çok amaçlı optimizasyon problemleri.

Genetik Algoritma Elitizim Avantaj-Dezavantaj

Avantajlar:

- **Çözüm kalitesi garantisi:** En iyi bulunan çözüm asla kaybolmaz.
- **Daha hızlı yakınsama:** İyi çözümler etrafında arama yoğunlaşır.
- **Kararlılık:** Algoritma çıktısı daha tutarlı ve tekrarlanabilir olur.
- **Teorik destek:** Elitizimli GA'ların global optimuma yakınsama olasılığı artar (markov zinciri analizleri ile kanıtlanmıştır).

Dezavantajlar:

- **Erken yakınsama riski:** Elit bireyler popülasyona hakim olursa, çeşitlilik azalır ve algoritma yerel optimumda sıkışabilir.
- **Hesaplama maliyeti:** Elitlerin saklanması ve karşılaştırılması ek bellek/işlem gerektirebilir (genellikle ihmal edilebilir düzeyde).
- **Parametre duyarlılığı:** Elit sayısı (k) çok yüksek seçilirse keşif (exploration) zayıflar; çok düşük seçilirse fayda azalır.

Pratik Kural: Elit sayısı genellikle popülasyon büyüklüğünün %1-%5'i arasında seçilir. Örneğin, popülasyon = 100 ise $k = 1 \sim 5$ idealdir.